

## 1. BUILT-IN FUNCTIONS (MUST KNOW)

### ★ What are Built-in Functions ?

Built-in functions are pre-defined in Python.

They are always available to use.

#### ① len()

Returns the number of items in an object.

Example :

```
s = "Python"
print(len(s))
```

Output :

6

#### ② sorted()

Returns a new sorted list from the items.

Example :

```
nums = [4, 1, 7, 2]
print(sorted(nums))
```

Output :

[1, 2, 4, 7]

#### ③ reversed()

Returns a reverse iterator of the sequence.

Example :

```
nums = [1, 2, 3, 4]
print(list(reversed(nums)))
```

Output :

[4, 3, 2, 1]

#### ④ sum()

Returns the sum of all items.

Example :

```
nums = [1, 2, 3, 4, 5]
print(sum(nums))
```

Output :

15

#### ⑤ max()

Returns the largest item.

Example :

```
nums = [10, 20, 5, 30]
print(max(nums))
```

Output :

30

#### ⑥ min()

Returns the smallest item.

Example :

```
nums = [10, 20, 5, 30]
print(min(nums))
```

Output :

5

#### ⑦ all()

Returns True if all elements are True.

Example :

```
vals = [True, True, 1, 3]
print(all(vals))
```

Output :

True

#### ⑧ any()

Returns True if any element is True.

Example :

```
vals = [0, False, "", 5]
print(any(vals))
```

Output :

True

#### ⑨ enumerate()

Returns index and value as a pair.

Example :

```
for i, ch in enumerate("abc"):
    print(i, ch)
```

Output :

```
0 a
1 b
2 c
```

#### ⑩ zip()

Combines multiple iterables element-wise.

Example :

```
a = [1, 2, 3]
b = ["a", "b", "c"]
print(list(zip(a, b)))
```

Output :

```
[(1, 'a'),
 (2, 'b'),
 (3, 'c')]
```

#### ⑪ map()

Applies a function to all items.

Example :

```
nums = [1, 2, 3, 4]
res = list(map(lambda x: x*2, nums))
print(res)
```

Output :

[2, 4, 6, 8]

#### ⑫ filter()

Filters items using a function.

Example :

```
nums = [1, 2, 3, 4, 5, 6]
res = list(filter(lambda x: x%2==0, nums))
print(res)
```

Output :

[2, 4, 6]



## 2. DECORATORS & GENERATORS

### ★ DECORATORS

Decorators allow us to modify the behavior of a function without permanently changing it.

#### ① Basic Decorator

|                                                                                                                                                                                                                                |                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <pre>def my_decorator(func):     def wrapper():         print("Before function call")         func()         print("After function call")     return wrapper  @my_decorator def say_hello():     print("Hello, Python!")</pre> | <p>Output :</p> <p>Before function call</p> <p>Hello, Python!</p> <p>After function call</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|

#### ② Decorator with Arguments

|                                                                                                                                                                                                      |                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <pre>def repeat(num):     def decorator(func):         def wrapper():             for _ in range(num):                 func()         return wrapper  @repeat(3) def greet():     print("Hi!")</pre> | <p>Output :</p> <p>Hi!</p> <p>Hi!</p> <p>Hi!</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|

#### ③ Real World Use (Timing Example)

|                                                                                                                                                                                                                                                                                                                                               |                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <pre>import time  def timer(func):     def wrapper(*args, **kwargs):         start = time.time()         result = func(*args, **kwargs)         end = time.time()         print(f"Time taken: {end - start:.4f}s")         return result     return wrapper  @timer def add(a, b):     time.sleep(1)     return a + b  print(add(5, 7))</pre> | <p>Output :</p> <p>Time taken: 1.0005s</p> <p>12</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|

#### Key Takeaway :

Decorators are widely used in logging, authentication, authorization, caching, performance tracking etc.

### ★ GENERATORS

Generators allow us to iterate over a sequence without storing it in memory.

#### ① Using yield

|                                                                                                                                                                                             |                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <pre>def count(n):     i = 1     while i &lt;= n:         yield i         i += 1  gen = count(5) print(next(gen)) print(next(gen)) print(next(gen)) print(next(gen)) print(next(gen))</pre> | <p>Output :</p> <p>1</p> <p>2</p> <p>3</p> <p>4</p> <p>5</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|

#### ② Generator Expression

|                                                                                  |                                                               |
|----------------------------------------------------------------------------------|---------------------------------------------------------------|
| <pre>squares = (x*x for x in range(5))  for num in squares:     print(num)</pre> | <p>Output :</p> <p>0</p> <p>1</p> <p>4</p> <p>9</p> <p>16</p> |
|----------------------------------------------------------------------------------|---------------------------------------------------------------|

#### ③ Generator vs Return

| Return (List)                                                                                                                                        | Yield (Generator)                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <pre>def get_list(n):     res = []     for i in range(n):         res.append(i)     return res  lst = get_list(1000000) # Stores all in memory</pre> | <pre>def get_gen(n):     for i in range(n):         yield i  gen = get_gen(1000000) # Generates one by one</pre> |

#### ④ Memory Advantage

- Generators produce items on-the-fly.
- They are memory efficient.
- Ideal for large datasets and streams.

#### Key Takeaway :

Use decorators to extend functionality.  
Use generators to handle large data efficiently.



## 3. ITERATORS, COMPREHENSIONS & COLLECTIONS

### ★ ITERATORS

An iterator is an object that can be iterated upon, meaning you can traverse through all the values.

```
it = iter([10, 20, 30])
print(next(it))    # 10
print(next(it))    # 20
print(next(it))    # 30
# StopIteration (if next() called again)
```

Output :

### ★ ITERATOR vs GENERATOR

| Iterator                       | Generator                |
|--------------------------------|--------------------------|
| • Returns data one by one      | • Uses yield             |
| • Uses <code>__next__()</code> | • Generates on-the-fly   |
| • Uses more memory             | • Uses less memory       |
| • Example: list, tuple         | • Example: generator fn. |

### ★ COMPREHENSIONS

Comprehensions provide a concise way to create lists, sets and dictionaries.

#### ① List Comprehension

```
squares = [x*x for x in range(5)]
```

Output :  
[0, 1, 4, 9, 16]

#### ② Dictionary Comprehension

```
cube_dict = {x: x**3 for x in range(1, 5)}
```

Output :  
{1: 1, 2: 8, 3: 27, 4: 64}

#### ③ Set Comprehension

```
even_set = {x for x in range(10) if x % 2 == 0}
```

Output :  
{0, 2, 4, 6, 8}

### ★ COLLECTIONS MODULE (Important Classes)

The collections module provides specialized container datatypes.

#### ① Counter

Counts hashable objects.

```
from collections import Counter
cnt = Counter("aabbccda")
print(cnt)
```

Output :  
Counter({'b': 3, 'a': 2, 'c': 2, 'd': 1})

#### ② defaultdict

Provides a default value for a key that does not exist.

```
from collections import defaultdict
dd = defaultdict(int)
dd['a'] += 1
dd['b'] += 2
print(dd)
```

Output :  
defaultdict(<class 'int'>, {'a': 1, 'b': 2})

#### ③ deque

Double ended queue, supports fast append and pop from both ends.

```
from collections import deque
dq = deque([1, 2, 3])
dq.appendleft(0)    # [0, 1, 2, 3]
dq.append(4)        # [0, 1, 2, 3, 4]
dq.pop()            # 4
dq.popleft()        # 0
print(dq)
```

Output :  
deque([1, 2, 3])

#### NESTED LIST COMPREHENSION

```
matrix = [[i*j for j in range(3)] for i in range(3)]
print(matrix)
```

Output :  
[[0, 0, 0], [0, 1, 2], [0, 2, 4]]

#### CONDITIONAL COMPREHENSION

```
nums = [x for x in range(10) if x % 2 == 0 and x > 2]
print(nums)
```

Output :  
[4, 6, 8]

**Key Takeaway :** Iterators and comprehensions help in writing clean, efficient code.  
Collections module provides powerful tools for real-world applications.



## 4. REGULAR EXPRESSIONS, JSON & DATETIME

### ★ REGULAR EXPRESSIONS (re MODULE)

Used to work with patterns in strings.

#### ① re.search()

Searches the pattern anywhere in the string.

```
import re
text = "Python is fun"
res = re.search(r"fun", text)
print(res.group())
```

Output :  
fun

#### ② re.match()

Matches the pattern at the beginning of the string.

```
text = "Python is fun"
res = re.match(r"Python", text)
print(res.group())
```

Output :  
Python

#### ③ re.findall()

Returns all non-overlapping matches.

```
text = "cat sat mat pat cat"
res = re.findall(r"cat", text)
print(res)
```

Output :  
['cat', 'cat']

#### ④ Common Patterns

| Pattern | Meaning                  |
|---------|--------------------------|
| .       | Any single character     |
| ^abc    | String starts with 'abc' |
| abc\$   | String ends with 'abc'   |
| [a-z]   | Any lowercase letter     |
| \d      | Any digit                |
| \s      | Any whitespace           |
| \w      | Any word character       |

### ★ JSON (json MODULE)

Used to work with JSON data.

#### ① Python → JSON (dumps)

```
import json
data = {"name": "Aman", "age": 25}
json_str = json.dumps(data, indent=2)
print(json_str)
```

Output :  
{  
 "name": "Aman",  
 "age": 25  
}

#### ② JSON → Python (loads)

```
import json
json_str = '{"name": "Aman", "age": 25}'
data = json.loads(json_str)
print(data["name"])
```

Output :  
Aman

#### ③ Read JSON from File

```
import json
with open("data.json", "r") as f:
    data = json.load(f)
print(data)
```

Output :  
{'name': 'Aman',  
'age': 25}

#### ④ Write JSON to File

```
import json
data = {"name": "Aman", "age": 25}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)
```

Output :  
File  
data.json  
created.

### ★ DATETIME (datetime MODULE)

Used to work with dates and times.

#### ① Current Date & Time

```
from datetime import datetime
now = datetime.now()
print(now)
```

Output :  
2024-05-20 14:35:22.123456

#### ② Formatting Date

```
from datetime import datetime
now = datetime.now()
formatted = now.strftime("%Y-%m-%d %H:%M:%S")
print(formatted)
```

Output :  
2024-05-20 14:35:22

#### ③ Parsing Date from String

```
from datetime import datetime
date_str = "2024-05-20 14:35:22"
dt = datetime.strptime(date_str, "%Y-%m-%d %H:%M:%S")
print(dt)
```

Output :  
2024-05-20 14:35:22

#### ④ Date Arithmetic

```
from datetime import datetime, timedelta
now = datetime.now()
next_day = now + timedelta(days=1)
print(next_day.strftime("%Y-%m-%d"))
```

Output :  
2024-05-21

**Key Takeaway :** Regex helps in pattern matching, JSON is used for data exchange, and datetime makes date & time operations easy.



# PYTHON HANDBOOK PART 3

@Aman\_views

## 5. MULTITHREADING & MULTIPROCESSING

### ★ MULTITHREADING (threading MODULE)

- Used for I/O-bound tasks.
- Threads share the same memory.

#### ① Basic Thread Example

```
import threading
```

```
def task(name):
```

```
    print(f"Hello from {name}")
```

Output :

Hello from Thread-1

Hello from Thread-2

```
t1 = threading.Thread(target=task, args=("Thread-1",))
```

```
t2 = threading.Thread(target=task, args=("Thread-2",))
```

```
t1.start()
```

```
t2.start()
```

```
t1.join()
```

```
t2.join()
```

#### ② Thread with Lock (Counter Example)

```
import threading
```

```
counter = 0
```

```
lock = threading.Lock()
```

```
def increment():
```

```
    global counter
```

```
    for _ in range(100000):
```

```
        with lock:
```

```
            counter += 1
```

Output :

200000

```
t1 = threading.Thread(target=increment)
```

```
t2 = threading.Thread(target=increment)
```

```
t1.start(); t2.start()
```

```
t1.join(); t2.join()
```

```
print(counter)
```

#### ③ ThreadPoolExecutor

```
from concurrent.futures import ThreadPoolExecutor
```

```
def work(n):
```

```
    import time
```

```
    time.sleep(1)
```

```
    return n * n
```

Output :

[1, 4, 9, 16, 25]

```
with ThreadPoolExecutor(max_workers=3) as executor:
```

```
    results = executor.map(work, [1, 2, 3, 4, 5])
```

```
    print(list(results))
```

### ★ MULTIPROCESSING (multiprocessing MODULE)

- Used for CPU-bound tasks.
- Processes have separate memory.

#### ① Basic Process Example

```
from multiprocessing import Process
```

```
def task(name):
```

```
    print(f"Hello from {name}")
```

Output :

Hello from Process-1

Hello from Process-2

```
p1 = Process(target=task, args=("Process-1",))
```

```
p2 = Process(target=task, args=("Process-2",))
```

```
p1.start()
```

```
p2.start()
```

```
p1.join()
```

```
p2.join()
```

#### ② ProcessPoolExecutor

```
from concurrent.futures import ProcessPoolExecutor
```

```
def square(n):
```

```
    return n * n
```

Output :

[1, 4, 9, 16, 25]

```
with ProcessPoolExecutor(max_workers=3) as executor:
```

```
    results = executor.map(square, [1, 2, 3, 4, 5])
```

```
    print(list(results))
```

#### ③ Threading vs Multiprocessing

| Feature       | Threading          | Multiprocessing     |
|---------------|--------------------|---------------------|
| Memory        | Shared             | Separate            |
| Best For      | I/O-bound tasks    | CPU-bound tasks     |
| Overhead      | Lower              | Higher              |
| Communication | Easy (shared data) | Need IPC / Queue    |
| GIL Impact    | Affected by GIL    | Not affected by GIL |

### ★ WHEN TO USE WHAT?

Use Multithreading when:

- Your task involves I/O operations (files, network, database, API calls etc.)
- You want to perform many tasks concurrently.
- Lightweight and fast to create.

Use Multiprocessing when:

- Your task involves heavy CPU computation.
- You need true parallelism.
- Each task runs independently.

**Key Takeaway :**

- Use multithreading for I/O-bound tasks to improve responsiveness.
- Use multiprocessing for CPU-bound tasks to utilize multiple cores effectively.